

SAA - ACTIVIDAD 6.6 - ALVARO FERNANDEZ

01. Regresión con Redes Neuronales en TensorFlow

Existen muchas definiciones para un [problema de regresión](#), pero en nuestro caso, vamos a simplificarlo a **predecir un número**.

Por ejemplo, podrías querer:

- Predecir el precio de venta de casas dada información sobre ellas (como el número de habitaciones, tamaño, número de baños).
- Predecir las coordenadas de un cuadro delimitador de un objeto en una imagen.
- Predecir el coste de un seguro médico para una persona dada su demografía (edad, sexo, género, raza).

En este notebook, vamos a establecer las bases de cómo tomar una muestra de entradas (estos son tus datos), construir una red neuronal para descubrir patrones en esas entradas y luego hacer una predicción (en forma de un número) basada en esas entradas.

Lo que vamos a cubrir

Específicamente, vamos a hacer las siguientes tareas con TensorFlow:

- Arquitectura de un modelo de regresión
- Formas de las entradas y salidas
 - **X**: características/datos (entradas)
 - **y**: etiquetas (salidas)
- Crear datos personalizados para visualizar y ajustar
- Pasos en el modelado
 - Crear un modelo
 - Compilar un modelo
 - Definir una función de pérdida
 - Configurar un optimizador
 - Crear métricas de evaluación
 - Ajustar un modelo (hacer que encuentre patrones en nuestros datos)
- Evaluar un modelo
 - Visualizar el modelo
 - Observar las curvas de entrenamiento
 - Comparar las predicciones con los valores reales (usando nuestras métricas de evaluación)
- Guardar un modelo (para poder usarlo más tarde)
- Cargar un modelo

Empieza a programar o a [crear código](#) con IA.

Arquitectura típica de una red neuronal de regresión

Hiperparámetro	Valor típico
Forma de la capa de entrada	Igual a la cantidad de características (por ejemplo, 3 para # habitaciones, # baños, # espacios para autos en la predicción de precio de casas)
Capa(s) oculta(s)	Específico del problema, mínimo = 1, máximo = ilimitado
Neuronas por capa oculta	Específico del problema, generalmente de 10 a 100
Forma de la capa de salida	Igual a la forma de la predicción deseada (por ejemplo, 1 para el precio de una casa)
Activación en las capas ocultas	Generalmente ReLU (unidad lineal rectificadora)
Activación en la salida	Ninguna, ReLU, logística/tanh
Función de pérdida	MSE (error cuadrático medio) o MAE (error absoluto medio)/Huber (combinación de MAE/MSE) si hay valores atípicos
Optimizador	SGD (descenso de gradiente estocástico), Adam

Tabla 1: Arquitectura típica de una red de regresión. Fuente: Adaptado de la página 293 del [libro Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow de Aurélien Géron](#)

 **Nota:** Un **hiperparámetro** en el aprendizaje automático es algo que un analista de datos o desarrollador puede configurar por sí mismo, mientras que un **parámetro** generalmente describe algo que un modelo aprende por sí mismo (un valor que no es configurado explícitamente por un analista).

```
# primero, vamos a importar librerías
import numpy as np
import matplotlib.pyplot as plt
```

```
import matplotlib.pyplot as plt

import tensorflow as tf
print(tf.__version__) # check the version (should be 2.x+)

import datetime
print(f"Última ejecución del notebook: {datetime.datetime.now()}")
```

2.20.0
Última ejecución del notebook: 2026-05-12 15:35:57.946184

Crear datos para visualizarlos y entrenar

Como estamos trabajando en un problema de **regresión** (predecir un número) vamos a crear algunos datos lineales para modelarlos

Formas de entrada y salida en regresión

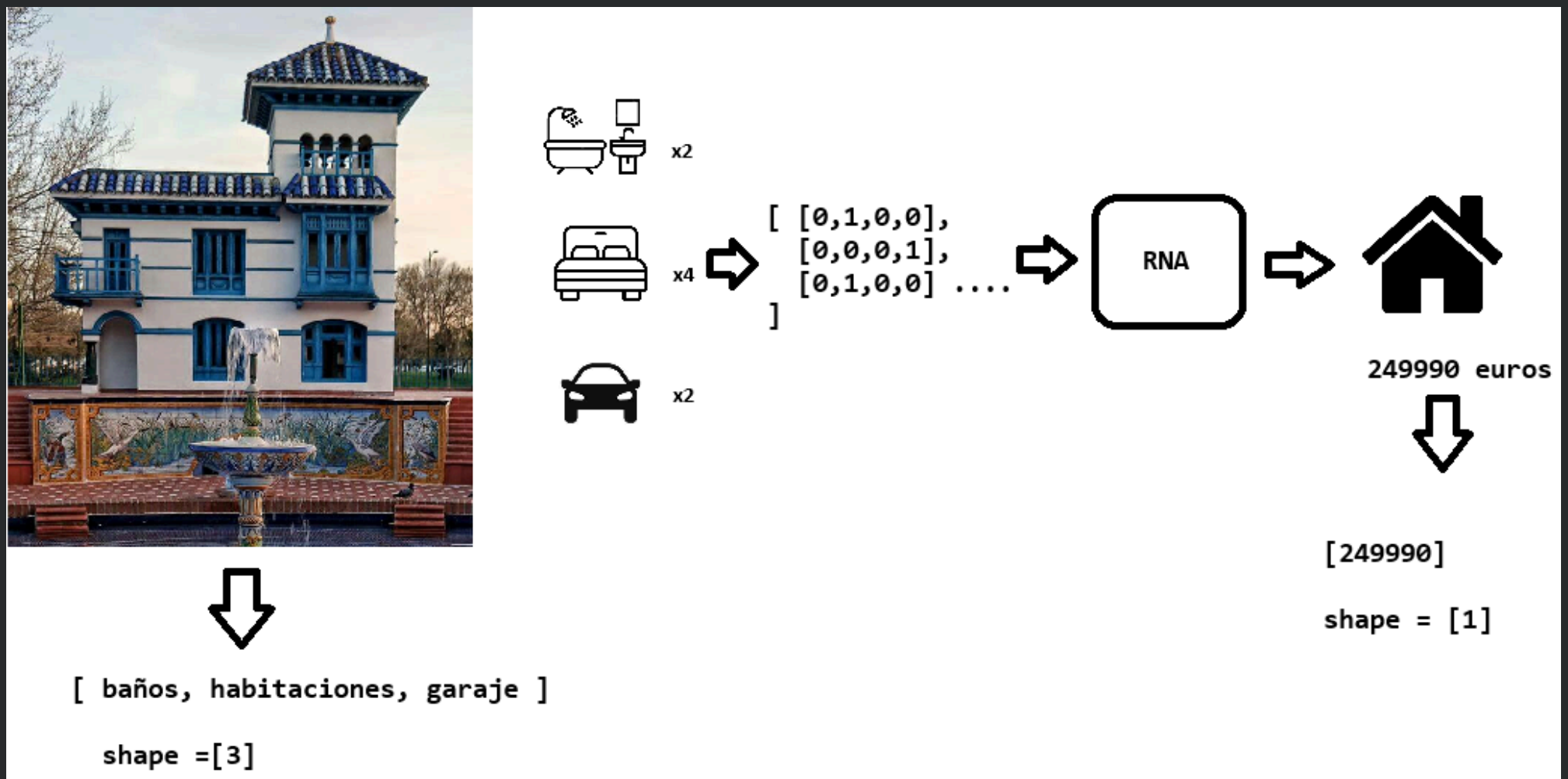
Uno de los conceptos más importantes al trabajar con redes neuronales son las formas de entrada y salida.

La **forma de entrada** (input shape) es la forma de tus datos que ingresan al modelo.

La **forma de salida** (output shape) es la forma de los datos que quieres que salgan de tu modelo.

Estas **formas** (shapes) variarán dependiendo del problema en el que estés trabajando.

Las redes neuronales aceptan números y generan números. Estos números generalmente se representan como tensores (o arrays).



Haz doble clic (o pulsa Intro) para editar

```
# Ejemplo de formas de entrada y salida
info_casas = tf.constant([[0,1,0,0], [0,0,0,1], [0,1,0,0]])
precio_casa = tf.constant([249990])

# visualiza los dos tensores, ¿Qué forma tienen?
print("info_casas:", info_casas)
print("Forma de info_casas:", info_casas.shape)
print("precio_casa:", precio_casa)
print("Forma de precio_casa:", precio_casa.shape)
```

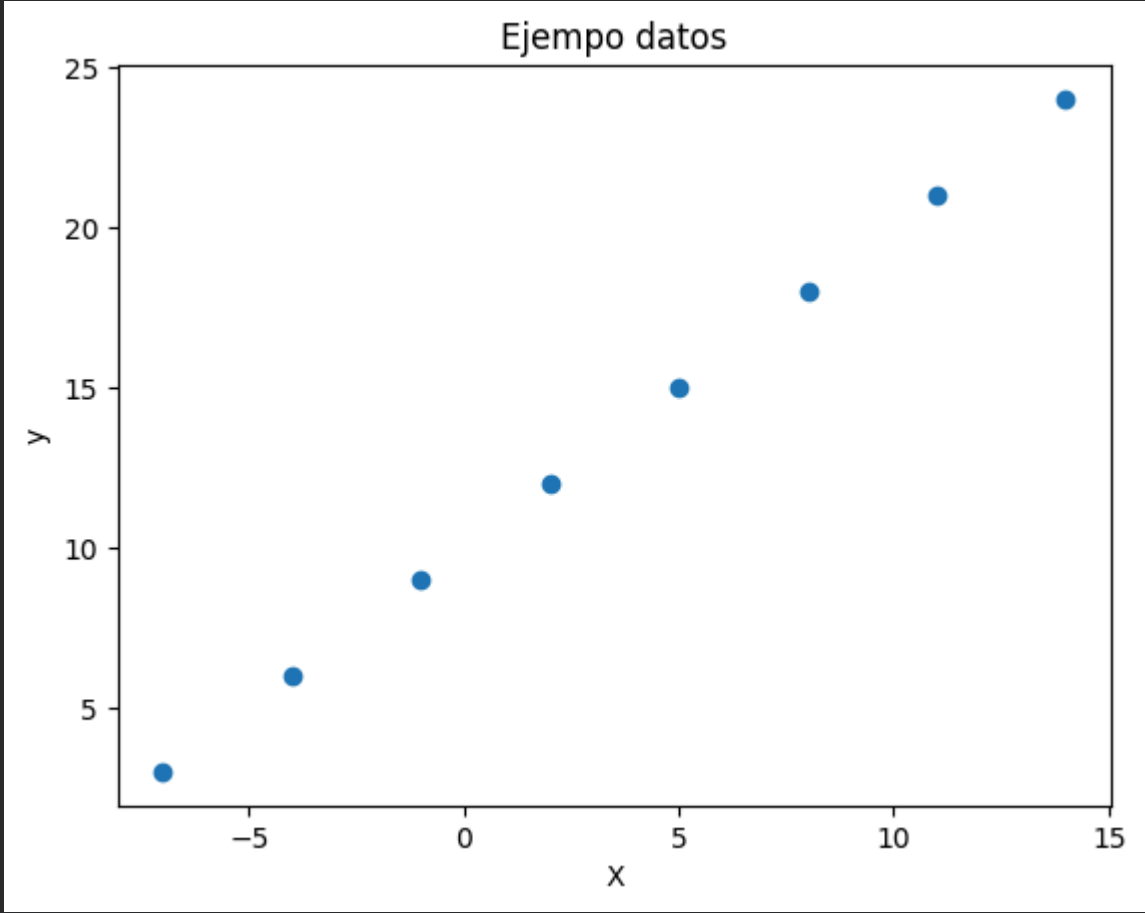
```
info_casas: tf.Tensor(
[[0 1 0 0]
 [0 0 0 1]
 [0 1 0 0]], shape=(3, 4), dtype=int32)
Forma de info_casas: (3, 4)
precio_casa: tf.Tensor([249990], shape=(1,), dtype=int32)
Forma de precio_casa: (1,)
```

generamos datos para visualizar

```
# Crea un tensor X de características con valores [-7.0, -4.0, -1.0, 2.0, 5.0, 8.0, 11.0, 14.0]
X = tf.constant([-7.0, -4.0, -1.0, 2.0, 5.0, 8.0, 11.0, 14.0])
```

```
# Crea un tensor y de características con valores [3.0, 6.0, 9.0, 12.0, 15.0, 18.0, 21.0, 24.0]
y = tf.constant([3.0, 6.0, 9.0, 12.0, 15.0, 18.0, 21.0, 24.0])

# Visualízalo con matplotlib
plt.scatter(X, y)
plt.xlabel("X")
plt.ylabel("y")
plt.title("Ejemplo datos")
plt.show()
```



Nuestro objetivo aquí será usar `X` para predecir `y`.

Entonces, nuestra **entrada** será `X` y nuestra **salida** será `y`.

Sabiendo esto, ¿qué crees que serán nuestras formas de entrada y salida?

Vamos a explorar estas variables.

```
# Coge una muestra del tensor X y otra del tensor y
x_muestra = X[0].shape
y_muestra = y[0].shape

# ¿Qué forma tienen?
print("Muestra X:" )
x_muestra
print("\nMuestra y:" )
y_muestra
```

```
Muestra X:

Muestra y:
TensorShape([])
```

- los dos resultados son tensores de rango 0 o escalares

▼

Pasos para modelar con TensorFlow

Ahora que sabemos qué datos tenemos, así como las formas de entrada y salida, veamos cómo construiríamos una red neuronal para modelarlos.

En TensorFlow, generalmente hay 3 pasos fundamentales para crear y entrenar un modelo.

1. **Crear un modelo** - ensamblar las capas de una red neuronal tu mismo (usando la [API Funcional](#) o la [API Secuencial](#)) o importar un modelo previamente construido (conocido como aprendizaje por transferencia).
2. **Compilar un modelo** - definir cómo debe medirse el rendimiento de un modelo (pérdida/métricas) y también cómo debe mejorar (optimizador).
3. **Ajustar un modelo** - permitir que el modelo intente encontrar patrones en los datos (¿cómo se relaciona `X` con `y`?).

Veamos estos pasos en acción usando la [API Secuencial de Keras](#) para construir un modelo para nuestros datos de regresión. Luego repasaremos cada uno de ellos.

```
# a partir de esta semilla:
```

```
tf.random.set_seed(42)

# crea un modelo secuencial con una capa densa de tan sólo una unidad
model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=[1])
])

# compila el modelo
# utilizando como función de pérdida y métrica "mae - mean absolute error", como optimizador "stochastic gradient descent"
# loss=, metrics=, optimizer=
model.compile(loss="mae",
              optimizer="SGD",
              metrics=["mae"])

# entrena el modelo utilizando la función fit durante 5 épocas
model.fit(X, y, epochs=5)

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
1/1 ━━━━━━━━━━━ 1s 597ms/step - loss: 14.2486 - mae: 14.2486
Epoch 2/5
1/1 ━━━━━━━━━━━ 0s 44ms/step - loss: 14.1161 - mae: 14.1161
Epoch 3/5
1/1 ━━━━━━━━━━━ 0s 41ms/step - loss: 13.9836 - mae: 13.9836
Epoch 4/5
1/1 ━━━━━━━━━━━ 0s 40ms/step - loss: 13.8511 - mae: 13.8511
Epoch 5/5
1/1 ━━━━━━━━━━━ 0s 42ms/step - loss: 13.7186 - mae: 13.7186
<keras.src.callbacks.history.History at 0x7a1724337d10>
```

Ya hemos entrenado nuestro modelo, ¿cómo crees que ha ido?

```
# Haz una predicción para el número 17
model.predict(tf.constant([17.0]))

1/1 ━━━━━━━━━━━ 0s 36ms/step
array([[ -0.6111416]], dtype=float32)
```

Parece ser que no ha predicho demasiado bien, la salida debería haber sido cerca de 27

Mejorando un modelo

Para mejorar nuestro modelo, podemos modificar casi todas las partes de los 3 pasos que tenemos para crear el modelo:

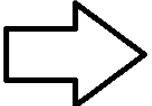
1. **Crear un modelo** - aquí podrías querer agregar más capas, aumentar el número de neuronas (unidades ocultas) dentro de cada capa, o cambiar las funciones de activación de cada capa.
2. **Compilar un modelo** - podrías elegir otra función de optimización o quizás cambiar la **tasa de aprendizaje** de la función de optimización.
3. **Ajustar un modelo** - podrías entrenar el modelo más **épocas** o usar más datos (darle más ejemplos al modelo para que aprenda).

MODELO PEQUEÑO:

```
# 1. Create the model (specified to your problem)
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(4, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

# 2. Compile the model
model.compile(loss=tf.keras.losses.BinaryCrossentropy(),
              optimizer=tf.keras.optimizers.Adam(lr=0.001),
              metrics=["accuracy"])

# 3. Fit the model
model.fit(X_train_subset, y_train_subset, epochs=5)
```



MODELO MÁS GRANDE:

```
# 1. Create the model (specified to your problem)
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

# 2. Compile the model
model.compile(loss=tf.keras.losses.BinaryCrossentropy(),
              optimizer=tf.keras.optimizers.Adam(lr=0.0001),
              metrics=["accuracy"])

# 3. Fit the model
model.fit(X_train_full, y_train_full, epochs=100)
```

Existen muchas formas diferentes de mejorar potencialmente una red neuronal. Algunas de las más comunes incluyen: aumentar el número de capas (haciendo la red más profunda), aumentar el número de unidades ocultas (haciendo la red más ancha) y cambiar la tasa de aprendizaje. Como estos valores son modificables por el humano, se les llama [hiperparámetros](#) y la práctica de tratar de encontrar los mejores hiperparámetros se conoce como [ajuste de hiperparámetros](#).

▼ Formas comunes de mejorar un modelo:

- Añadir capas
- Incrementar el número de unidades ocultas

- Cambiar las funciones de activación
- Cambiar la función de optimización
- Cambiar la tasa de aprendizaje (porque puedes alterar cada una de las capas, son hiperparámetros)
- Entrenar con más datos
- Entrenar por más tiempo (más épocas)

✓ Ahora tu: Intenta entrenar más épocas

Nuestro primer instinto es entrenar por más tiempo. ¿Da siempre esto el resultado esperado?

```
# reproduce el modelo anterior y entrena con 100 épocas
tf.random.set_seed(42)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=[1])
])

model.compile(loss="mae",
              optimizer="SGD",
              metrics=["mae"])

model.fit(X, y, epochs=100)
```

```
2/2 ██████████ 0s 50ms/step - loss: 12.1816 - mae: 12.1816
Epoch 73/100
2/2 ██████████ 0s 30ms/step - loss: 11.5017 - mae: 11.5017
Epoch 74/100
2/2 ██████████ 0s 29ms/step - loss: 12.1703 - mae: 12.1703
Epoch 75/100
2/2 ██████████ 0s 29ms/step - loss: 11.4895 - mae: 11.4895
Epoch 76/100
2/2 ██████████ 0s 30ms/step - loss: 12.1590 - mae: 12.1590
Epoch 77/100
2/2 ██████████ 0s 29ms/step - loss: 11.4774 - mae: 11.4774
Epoch 78/100
2/2 ██████████ 0s 30ms/step - loss: 12.1477 - mae: 12.1477
Epoch 79/100
2/2 ██████████ 0s 28ms/step - loss: 11.4652 - mae: 11.4652
Epoch 80/100
2/2 ██████████ 0s 29ms/step - loss: 12.1364 - mae: 12.1364
Epoch 81/100
2/2 ██████████ 0s 28ms/step - loss: 11.4530 - mae: 11.4530
Epoch 82/100
2/2 ██████████ 0s 29ms/step - loss: 12.1250 - mae: 12.1250
Epoch 83/100
2/2 ██████████ 0s 34ms/step - loss: 11.4408 - mae: 11.4408
Epoch 84/100
2/2 ██████████ 0s 33ms/step - loss: 12.1137 - mae: 12.1137
Epoch 85/100
2/2 ██████████ 0s 31ms/step - loss: 11.4287 - mae: 11.4287
Epoch 86/100
2/2 ██████████ 0s 41ms/step - loss: 12.1024 - mae: 12.1024
Epoch 87/100
2/2 ██████████ 0s 31ms/step - loss: 11.4165 - mae: 11.4165
Epoch 88/100
2/2 ██████████ 0s 36ms/step - loss: 12.0911 - mae: 12.0911
Epoch 89/100
2/2 ██████████ 0s 32ms/step - loss: 11.4043 - mae: 11.4043
Epoch 90/100
2/2 ██████████ 0s 22ms/step - loss: 12.0798 - mae: 12.0798
Epoch 91/100
2/2 ██████████ 0s 23ms/step - loss: 11.3921 - mae: 11.3921
Epoch 92/100
2/2 ██████████ 0s 23ms/step - loss: 12.0685 - mae: 12.0685
Epoch 93/100
2/2 ██████████ 0s 24ms/step - loss: 11.3800 - mae: 11.3800
Epoch 94/100
2/2 ██████████ 0s 22ms/step - loss: 12.0572 - mae: 12.0572
Epoch 95/100
2/2 ██████████ 0s 24ms/step - loss: 11.3678 - mae: 11.3678
Epoch 96/100
2/2 ██████████ 0s 23ms/step - loss: 12.0459 - mae: 12.0459
Epoch 97/100
2/2 ██████████ 0s 23ms/step - loss: 11.8346 - mae: 11.8346
Epoch 98/100
2/2 ██████████ 0s 24ms/step - loss: 10.6864 - mae: 10.6864
Epoch 99/100
2/2 ██████████ 0s 23ms/step - loss: 12.6974 - mae: 12.6974
Epoch 100/100
2/2 ██████████ 0s 23ms/step - loss: 11.2423 - mae: 11.2423
<keras.src.callbacks.history.History at 0x7a17a888a390>
```

Habrás notado que el valor de la pérdida ha disminuido (y sigue disminuyendo a medida que aumenta el número de épocas).

¿Podremos hacer una predicción fiable con nuestro modelo?

¿Qué tal si intentamos predecir nuevamente con 17.0?

```
# Predice de nuevo la salida de la entrada 17. ¿Ha mejorado?
model.predict(tf.constant([17.0]))

1/1 ----- 0s 97ms/step
array([[29.314669]], dtype=float32)
```

Mucho mejor! Estamos más cerca, pero no lo suficiente... ¿o sí?

¿cómo lo evaluamos?

✓ Evaluando un modelo

Un flujo de trabajo típico que seguirás al construir redes neuronales es:

Construir un modelo -> evaluarlo -> construir (ajustar) un modelo -> evaluarlo -> construir (ajustar) un modelo -> evaluarlo...

Los ajustes vienen de no construir un modelo desde cero, sino de modificar uno existente.

Visualizar

Probablemente aprenderás más observando algo (haciendo) que pensando en algo.

Es una buena idea visualizar:

- **Los datos** - ¿con qué datos estás trabajando? ¿Cómo se ven?
- **El propio modelo** - ¿cómo se ve la arquitectura? ¿Cuáles son las diferentes formas?
- **El entrenamiento de un modelo** - ¿cómo rinde un modelo mientras aprende?
- **Las predicciones de un modelo** - ¿cómo se alinean las predicciones de un modelo con la verdad real (las etiquetas originales)?

Empecemos visualizando el modelo.

Pero primero, crearemos un conjunto de datos un poco más grande y un nuevo modelo que podamos usar (será el mismo que antes, pero más práctica siempre es mejor).

```
# crea un array X en numpy con el método arange para generar números desde -100 a 100, de 4 en 4
X = np.arange(-100, 100, 4)
X

array([-100,  -96,  -92,  -88,  -84,  -80,  -76,  -72,  -68,  -64,  -60,
        -56,  -52,  -48,  -44,  -40,  -36,  -32,  -28,  -24,  -20,  -16,
         -12,   -8,   -4,   0,    4,    8,   12,   16,   20,   24,   28,
          32,   36,   40,   44,   48,   52,   56,   60,   64,   68,   72,
          76,   80,   84,   88,   92,   96])
```

```
# creamos datos para las etiquetas (X+10)
y = X + 10
y

array([-90, -86, -82, -78, -74, -70, -66, -62, -58, -54, -50, -46, -42,
       -38, -34, -30, -26, -22, -18, -14, -10,  -6,  -2,   2,   6,  10,
        14,  18,  22,  26,  30,  34,  38,  42,  46,  50,  54,  58,  62,
        66,  70,  74,  78,  82,  86,  90,  94,  98, 102, 106])
```

✓ Dividir los datos en conjunto de entrenamiento/prueba

Uno de los pasos más comunes e importantes en un proyecto de aprendizaje automático es la creación de un conjunto de entrenamiento y un conjunto de prueba (y cuando es necesario, un conjunto de validación).

Cada conjunto tiene un propósito específico:

- **Conjunto de entrenamiento** *training*: el modelo aprende de estos datos, que suelen representar el 70-80% del total de datos disponibles (como los materiales de estudio que repasas durante el semestre).
- **Conjunto de validación** *validation*: el modelo se ajusta con estos datos, que suelen representar el 10-15% del total de datos disponibles (como el examen de práctica que tomas antes del examen final).
- **Conjunto de prueba** *test*: el modelo se evalúa con estos datos para comprobar lo que ha aprendido, suelen representar el 10-15% del total de datos disponibles (como el examen final que tomas al final del semestre).

Por ahora, solo utilizaremos un conjunto de entrenamiento y uno de prueba. Esto significa que tendremos un conjunto de datos para que nuestro modelo aprenda y otro para evaluarlo.

Podemos crearlos dividiendo nuestros arrays **X** y **y**.

🔑 **Nota:** Al tratar con datos del mundo real, este paso generalmente se realiza al comienzo de un proyecto (el conjunto de prueba siempre debe mantenerse separado de todos los demás datos). Queremos que nuestro modelo aprenda con los datos de entrenamiento y luego evaluarlo con los datos de prueba para obtener una indicación de qué tan bien **generaliza** a ejemplos no vistos.

```
# Comprueba cuantas muestras tenemos
print("Número de muestras:", len(X))
```

Número de muestras: 50

```
# Utiliza slicing para dividir el conjunto de datos en entrenamiento (80%) y test (20%)
n_train = int(len(X) * 0.8)

X_train, y_train = X[:n_train], y[:n_train]
X_test, y_test = X[n_train:], y[n_train:]

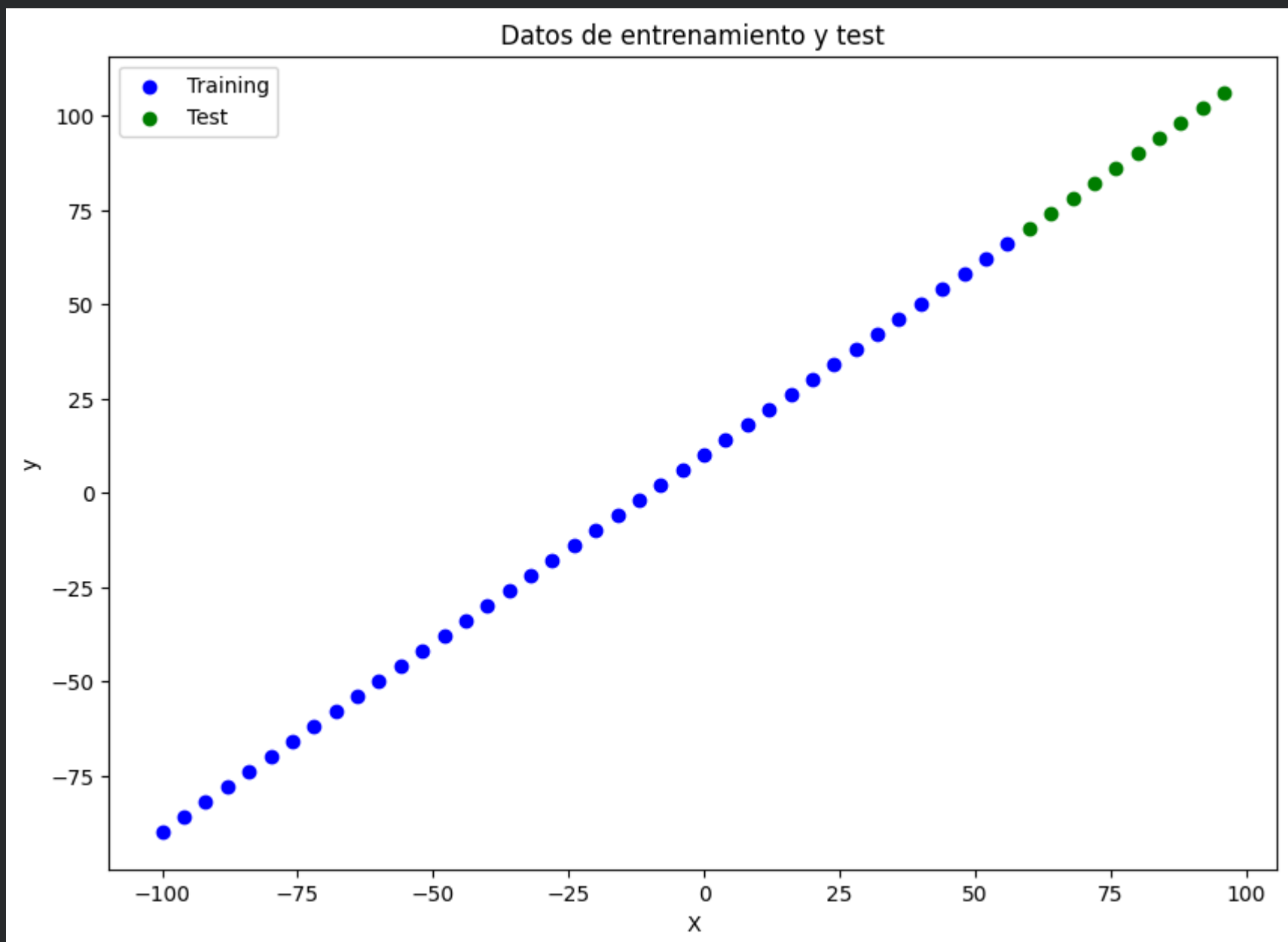
print("Tamaño train:", len(X_train), "\nTamaño test:", len(X_test))
```

Tamaño train: 40
Tamaño test: 10

Visualizando los datos

Ahora que tenemos los datos divididos, es buena idea visualizarlos. Los pintamos con colores diferentes

```
# visualiza con matplotlib los datos. Utiliza el color azul para los datos de training y el color verde para los datos de testing
plt.figure(figsize=(10, 7))
plt.scatter(X_train, y_train, c="b", label="Training")
plt.scatter(X_test, y_test, c="g", label="Test")
plt.xlabel("X")
plt.ylabel("y")
plt.title("Datos de entrenamiento y test")
plt.legend()
plt.show()
```



```
# copia el modelo que hemos creado anteriormente y comenta la llamada a la función fit
tf.random.set_seed(42)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(1)
])

model.compile(loss="mae",
              optimizer="SGD",
```

```
metrics=["mae"]])
```

```
# model.fit(X_train, y_train, epochs=50)
```

Visualizando el modelo

Después de construir un modelo, es posible que quieras echarle un vistazo (especialmente si no has construido muchos antes).

Puedes ver las capas y las formas de tu modelo llamando a `summary()` sobre él.

 **Nota:** Visualizar un modelo es particularmente útil cuando te encuentras con desajustes en las formas de entrada y salida.

```
# Invoca a la función summary
model.summary()

# No funciona del todo porque el modelo o nó está construido (build) o no está entrenado (fit)
# es decir, sabe el número de capas, pero no sabe el número de parámetros
```

Model: "sequential_9"

Layer (type)	Output Shape	Param #
dense_9 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Para que el modelo se imprima correctamente con todos los parámetros especificados, necesitamos añadir la forma de la entrada [1], con esto keras puede calcular el número de parámetros de la red

```
# si indicamos la forma de entrada a la capa 1, summary sí que es capaz de calcular los parámetros de la red
tf.random.set_seed(42)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=[1])
])

model.compile(loss="mae",
              optimizer="SGD",
              metrics=["mae"])

model.summary()
```

Model: "sequential_16"

Layer (type)	Output Shape	Param #
dense_16 (Dense)	(None, 1)	2

Total params: 2 (8.00 B)
Trainable params: 2 (8.00 B)
Non-trainable params: 0 (0.00 B)

como ves en el warning, no es la forma idónea, pero es un avance.

Llamar a `summary()` en nuestro modelo nos muestra las capas que contiene, la forma de salida y el número de parámetros.

- **Total params** - número total de parámetros en el modelo.
- **Parámetros entrenables** (*trainable params*) - son los parámetros (patrones) que el modelo puede actualizar mientras se entrena.
- **Parámetros no entrenables** (*non-trainable params*) - estos parámetros no se actualizan durante el entrenamiento (esto es típico cuando se utilizan patrones ya aprendidos de otros modelos durante el aprendizaje por transferencia).

 **Recurso:** Para una visión más profunda de los parámetros entrenables dentro de una capa, consulta el [video introductorio al aprendizaje profundo de MIT](#).

 **Ejercicio:** Intenta experimentar con el número de unidades ocultas en la capa `Dense` (por ejemplo, `Dense(2)`, `Dense(3)`). ¿Cómo cambia esto los parámetros Totales/Entrenables? Investiga qué está causando el cambio.

Por ahora, todo lo que necesitas saber sobre estos parámetros es que representan patrones que el modelo puede aprender de los datos.

Vamos a entrenar nuestro modelo con los datos de entrenamiento.

```
# Probamos por ejemplo con el valor 3
tf.random.set_seed(33)
```



```
model_otro = tf.keras.Sequential([
    tf.keras.layers.Dense(3, input_shape=[1])
])

model_otro.compile(loss="mae",
                   optimizer="SGD",
                   metrics=["mae"])

model_otro.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape`/`input\_dim`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_15"

Layer (type)	Output Shape	Param #
dense_15 (Dense)	(None, 3)	6

Total params: 6 (24.00 B)
Trainable params: 6 (24.00 B)
Non-trainable params: 0 (0.00 B)

✓ Parece que modificar el valor de **Dense** multiplica por el valor que le pongamos el numero de parametros

```
# entrena ahora el modelo durante 50 épocas
model.fit(X_train, y_train, epochs=50)
```

2/2 ██████████ 0s 24ms/step - loss: 9.6326 - mae: 9.6326
Epoch 23/50
2/2 ██████████ 0s 24ms/step - loss: 7.9566 - mae: 7.9566
Epoch 24/50
2/2 ██████████ 0s 23ms/step - loss: 9.6197 - mae: 9.6197
Epoch 25/50
2/2 ██████████ 0s 24ms/step - loss: 7.9380 - mae: 7.9380
Epoch 26/50
2/2 ██████████ 0s 24ms/step - loss: 9.6067 - mae: 9.6067
Epoch 27/50
2/2 ██████████ 0s 24ms/step - loss: 7.9193 - mae: 7.9193
Epoch 28/50
2/2 ██████████ 0s 24ms/step - loss: 9.5938 - mae: 9.5938
Epoch 29/50
2/2 ██████████ 0s 24ms/step - loss: 7.9006 - mae: 7.9006
Epoch 30/50
2/2 ██████████ 0s 25ms/step - loss: 9.5809 - mae: 9.5809
Epoch 31/50
2/2 ██████████ 0s 26ms/step - loss: 7.8819 - mae: 7.8819
Epoch 32/50
2/2 ██████████ 0s 25ms/step - loss: 9.5679 - mae: 9.5679
Epoch 33/50
2/2 ██████████ 0s 24ms/step - loss: 7.8632 - mae: 7.8632
Epoch 34/50
2/2 ██████████ 0s 25ms/step - loss: 9.5550 - mae: 9.5550
Epoch 35/50
2/2 ██████████ 0s 29ms/step - loss: 7.8445 - mae: 7.8445
Epoch 36/50
2/2 ██████████ 0s 24ms/step - loss: 9.5420 - mae: 9.5420
Epoch 37/50
2/2 ██████████ 0s 24ms/step - loss: 7.8258 - mae: 7.8258
Epoch 38/50
2/2 ██████████ 0s 23ms/step - loss: 9.5291 - mae: 9.5291
Epoch 39/50
2/2 ██████████ 0s 25ms/step - loss: 7.8071 - mae: 7.8071
Epoch 40/50
2/2 ██████████ 0s 25ms/step - loss: 9.5162 - mae: 9.5162
Epoch 41/50
2/2 ██████████ 0s 23ms/step - loss: 7.7885 - mae: 7.7885
Epoch 42/50
2/2 ██████████ 0s 24ms/step - loss: 9.5032 - mae: 9.5032
Epoch 43/50
2/2 ██████████ 0s 25ms/step - loss: 7.7698 - mae: 7.7698
Epoch 44/50
2/2 ██████████ 0s 24ms/step - loss: 9.4903 - mae: 9.4903
Epoch 45/50
2/2 ██████████ 0s 25ms/step - loss: 7.7511 - mae: 7.7511
Epoch 46/50
2/2 ██████████ 0s 25ms/step - loss: 9.4774 - mae: 9.4774
Epoch 47/50
2/2 ██████████ 0s 24ms/step - loss: 7.7324 - mae: 7.7324
Epoch 48/50
2/2 ██████████ 0s 26ms/step - loss: 9.4644 - mae: 9.4644
Epoch 49/50
2/2 ██████████ 0s 24ms/step - loss: 7.9016 - mae: 7.9016
Epoch 50/50
2/2 ██████████ 0s 25ms/step - loss: 8.5758 - mae: 8.5758
<keras.src.callbacks.history.History at 0x7a16fc7659d0>

```
# invoca a la función summary ¿Qué diferencia observas?
model.summary()
```

Model: "sequential_16"

Layer (type)	Output Shape	Param #
dense_16 (Dense)	(None, 1)	2

Total params: 4 (20.00 B)

Trainable params: 2 (8.00 B)

Non-trainable params: 0 (0.00 B)

Optimizer params: 2 (12.00 B)

Observa cómo la red ahora tiene 4 parámetros, 2 más que en el caso anterior. Estos son los parámetros del optimizador

Los **optimizer params** (parámetros del optimizador) son los valores configurables que controlan cómo un optimizador ajusta los pesos del modelo durante el entrenamiento. Estos incluyen parámetros como la **tasa de aprendizaje** (learning rate), que define el tamaño de los pasos que el optimizador da al actualizar los pesos, y otros como el **momento** (momentum), que ayuda a acelerar la convergencia.

Visualizando las predicciones

Ahora que tenemos un modelo entrenado, vamos a visualizar algunas predicciones.

Para visualizar predicciones, siempre es una buena idea graficarlas contra las etiquetas reales.

A menudo verás esto en la forma de `y_test` vs. `y_pred` (etiquetas reales vs. predicciones).

Primero, haremos algunas predicciones sobre los datos de prueba (`X_test`), recuerda que el modelo nunca ha visto los datos de prueba.

```
# realiza las predicciones sobre los datos de test
y_preds = model.predict(X_test)
y_preds
```

```
1/1 ----- 0s 39ms/step
array([[53.241734],
       [56.743225],
       [60.244717],
       [63.74621 ],
       [67.247696],
       [70.74919 ],
       [74.25068 ],
       [77.75217 ],
       [81.25366 ],
       [84.75515 ]], dtype=float32)
```

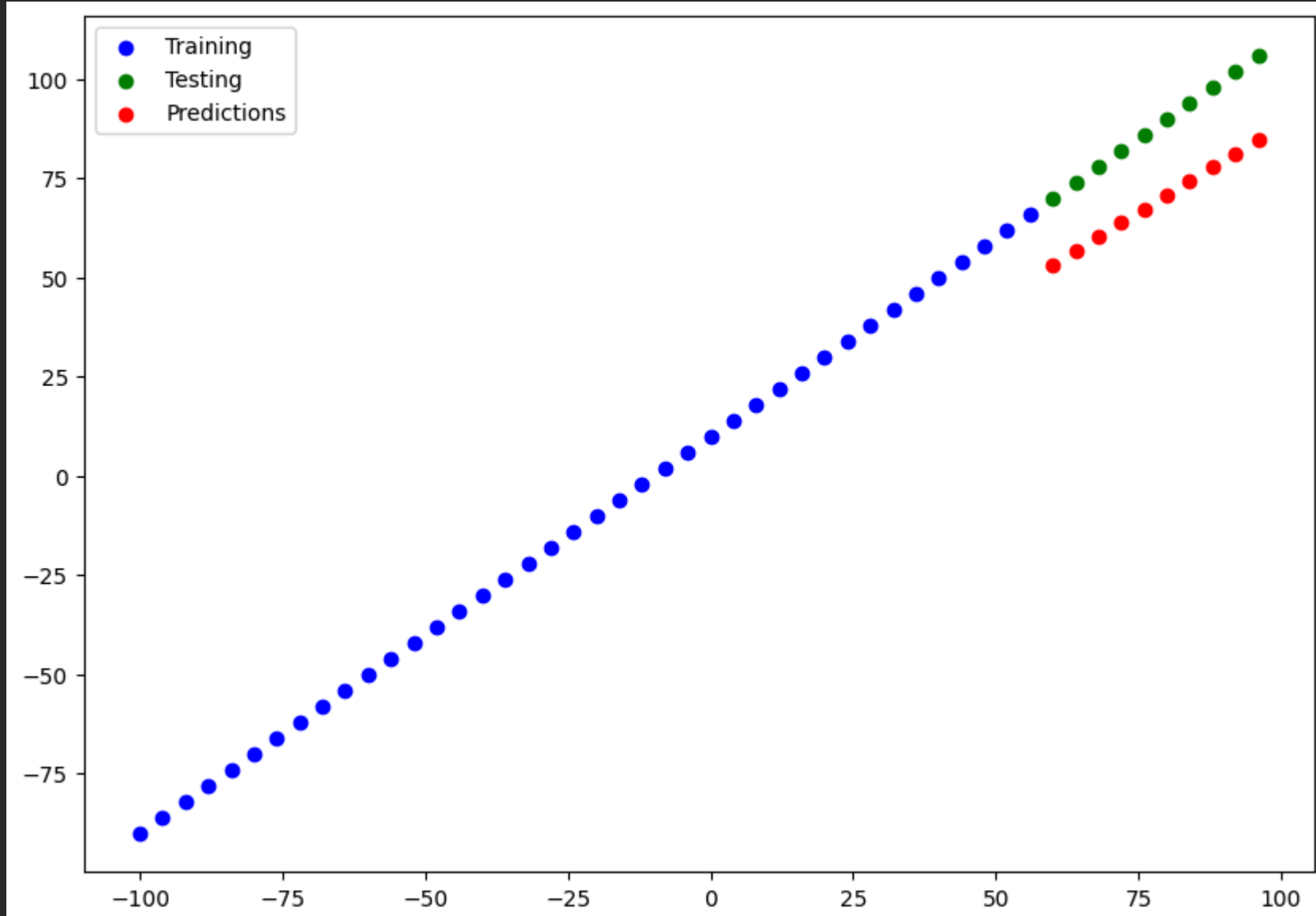
Bien, obtenemos una lista de números, pero ¿cómo se comparan con las etiquetas reales?

Vamos a construir una función de plot para averiguarlo.

```
def plot_predictions(train_data,
                     train_labels,
                     test_data,
                     test_labels,
                     predictions):
    """
    Visualiza datos de entrenamiento y test y compara predicciones
    """
    plt.figure(figsize=(10, 7))
    # training en azul
    plt.scatter(train_data, train_labels, c="b", label="Training")
    # test en verde
    plt.scatter(test_data, test_labels, c="g", label="Testing")
    # predicciones en rojo
    plt.scatter(test_data, predictions, c="r", label="Predictions")

    plt.legend();
```

```
# usando la función anterior, visualiza los datos de prueba, de test y las predicciones
plot_predictions(train_data=X_train,
                 train_labels=y_train,
                 test_data=X_test,
                 test_labels=y_test,
                 predictions=y_preds)
```



¿Cómo crees que resultó?

- De la gráfica podemos ver que nuestras predicciones no son correctas, pero tampoco son desacertadas
- Hay un margen de error aceptable y comprensible

✓ Evaluando las predicciones

Además de las visualizaciones, las métricas de evaluación son tu mejor opción alternativa para evaluar tu modelo.

Dependiendo del problema en el que estés trabajando, diferentes modelos tienen diferentes métricas de evaluación.

Dos de las principales métricas utilizadas para problemas de regresión son:

- **Error absoluto medio (MAE)** - la diferencia media entre cada una de las predicciones.
- **Error cuadrático medio (MSE)** - la diferencia media al cuadrado de las predicciones (se utiliza si los errores más grandes son más perjudiciales que los más pequeños).

Cuanto más bajos sean estos valores, mejor.

También puedes usar `model.evaluate()`, que devolverá la pérdida del modelo así como cualquier métrica configurada durante el paso de compilación.

```
# invoca al método evaluate pasando como parámetros X_test e y_test
model.evaluate(X_test, y_test)
```

```
1/1 — 0s 306ms/step - loss: 19.0016 - mae: 19.0016
[19.001556396484375, 19.001556396484375]
```

En nuestro caso, como usamos MAE como función de pérdida y también como métrica, `model.evaluate()` nos devuelve ambas.

TensorFlow también tiene funciones integradas para MSE y MAE.

Para muchas funciones de evaluación, el principio es el mismo: comparar las predicciones con las etiquetas reales.

```
# calcula el error absoluto medio con la función tf.keras.losses.MAE
tf.keras.losses.MAE(y_test, y_preds)
```

```
<tf.Tensor: shape=(10,), dtype=float32, numpy=
array([16.758266, 17.256775, 17.755283, 18.253792, 18.752304, 19.250809,
       19.749321, 20.247833, 20.746338, 21.24485 ], dtype=float32)>
```

¿Qué salida más rara, verdad? ¿No debería salir un único valor? Esto es porque `y_test` y `y_preds` no tienen la misma forma (shape)

```
#examina la forma de y_test y y_preds
print("Forma de y_test:", y_test.shape)
```

```
print("Forma de y_preds", y_preds.shape)
```

```
Forma de y_test: (10,)
Forma de y_preds: (10, 1)
```

Podemos solucionarlo usando `squeeze()`, que eliminará la dimensión `1` de nuestro tensor `y_preds`, haciendo que tenga la misma forma que `y_test`.

 **Nota:** Si estás comparando dos tensores, es importante asegurarse de que tengan las formas correctas (no siempre tendrás que manipular las formas, pero siempre presta atención, *muchos* errores son el resultado de tensores con formas incompatibles, especialmente en las formas de entrada y salida).

```
# utiliza el método squeeze para eliminar la dimensión 1
y_preds = tf.squeeze(y_preds)
print("Forma y_preds", y_preds.shape)
print("Forma y_test", y_test.shape)
```

```
Forma y_preds (10,)
Forma y_test (10,)
```

✓ - Ahora ambos tensores tienen la misma forma

```
# prueba de nuevo la función MAE
tf.keras.losses.MAE(y_test, y_preds)
```

```
<tf.Tensor: shape=(), dtype=float32, numpy=19.001556396484375>
```

```
# calculas también el MSE
tf.keras.losses.MSE(y_test, y_preds)
```

```
<tf.Tensor: shape=(), dtype=float32, numpy=363.1094055175781>
```

```
# vamos a crear funciones para nuestras evaluaciones
def mae(y_test, y_pred):
    return tf.keras.losses.MAE(y_test, y_pred)
```

```
def mse(y_test, y_pred):
    return tf.keras.losses.MSE(y_test, y_pred)
```

✓ Ejecutando experimentos para mejorar un modelo

Después de ver las métricas de evaluación y las predicciones que hace tu modelo, es probable que quieras mejorarlo.

Nuevamente, hay muchas formas diferentes de hacerlo, pero 3 de las principales son:

1. **Obtener más datos** - conseguir más ejemplos para que tu modelo entrene (más oportunidades para aprender patrones).
2. **Hacer tu modelo más grande (usar un modelo más complejo)** - esto podría significar más capas o más unidades ocultas en cada capa.
3. **Entrenar por más tiempo** - darle a tu modelo más oportunidades de encontrar patrones en los datos.

Como creamos nuestro propio conjunto de datos, podríamos fácilmente generar más datos, pero esto no siempre es el caso cuando trabajas con conjuntos de datos del mundo real.

Así que vamos a ver cómo podemos mejorar nuestro modelo utilizando las opciones 2 y 3.

Para hacerlo, construiremos 3 modelos y compararemos sus resultados:

1. `model_1` - igual que el modelo original, 1 capa, entrenado durante 100 épocas.
2. `model_2` - 2 capas, entrenado durante 100 épocas.
3. `model_3` - 2 capas, entrenado durante 500 épocas.

*** vamos a construir `model_1` **

```
# crear model_1 y entrenalo durante 100 épocas
tf.random.set_seed(42)
```

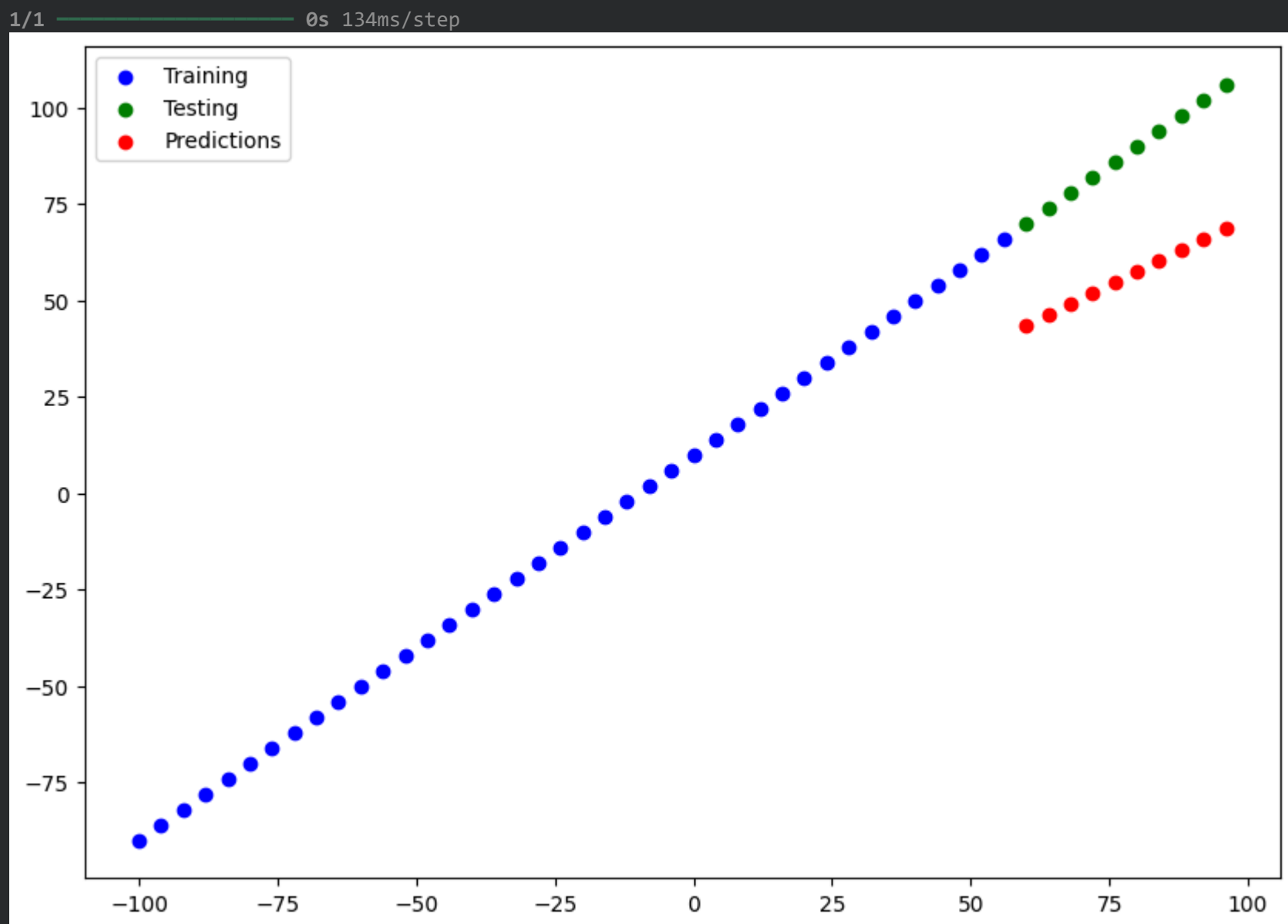
```
model_1 = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=[1])
])
```

```
model_1.compile(loss="mae",
                 optimizer="SGD",
                 metrics=["mae"])
```

```
#entrenamiento modelo 1
model_1.fit(X_train, y_train, epochs=100, verbose=0)

<keras.src.callbacks.history.History at 0x7a170defd610>
```

```
# visualiza las predicciones
y_preds_1 = tf.squeeze(model_1.predict(X_test))
plot_predictions(train_data=X_train,
                 train_labels=y_train,
                 test_data=X_test,
                 test_labels=y_test,
                 predictions=y_preds_1)
```



Calculemos las métricas

```
# visualiza la mae y la mse
mae_1 = mae(y_test, y_preds_1)
mse_1 = mse(y_test, y_preds_1)

print("METRICAS model_1:\nMAE:",mae_1.numpy()),"\nMSE:",mse_1.numpy())
```

```
METRICAS model_1:
MAE: 32.024178
MSE: 1037.4752
```

Modelo 2 (*model_2*) Esta vez agregaremos una capa densa adicional (así que ahora nuestro modelo tendrá 2 capas) manteniendo todo lo demás igual.

```
# crea ahora model_2, pero con una capa adicional de una unidad, entrénalo también durante 100 épocas
tf.random.set_seed(42)

model_2 = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=[1]),
    tf.keras.layers.Dense(1)
])

model_2.compile(loss="mae",
               optimizer="SGD",
               metrics=["mae"])

model_2.fit(X_train, y_train, epochs=100, verbose=0)

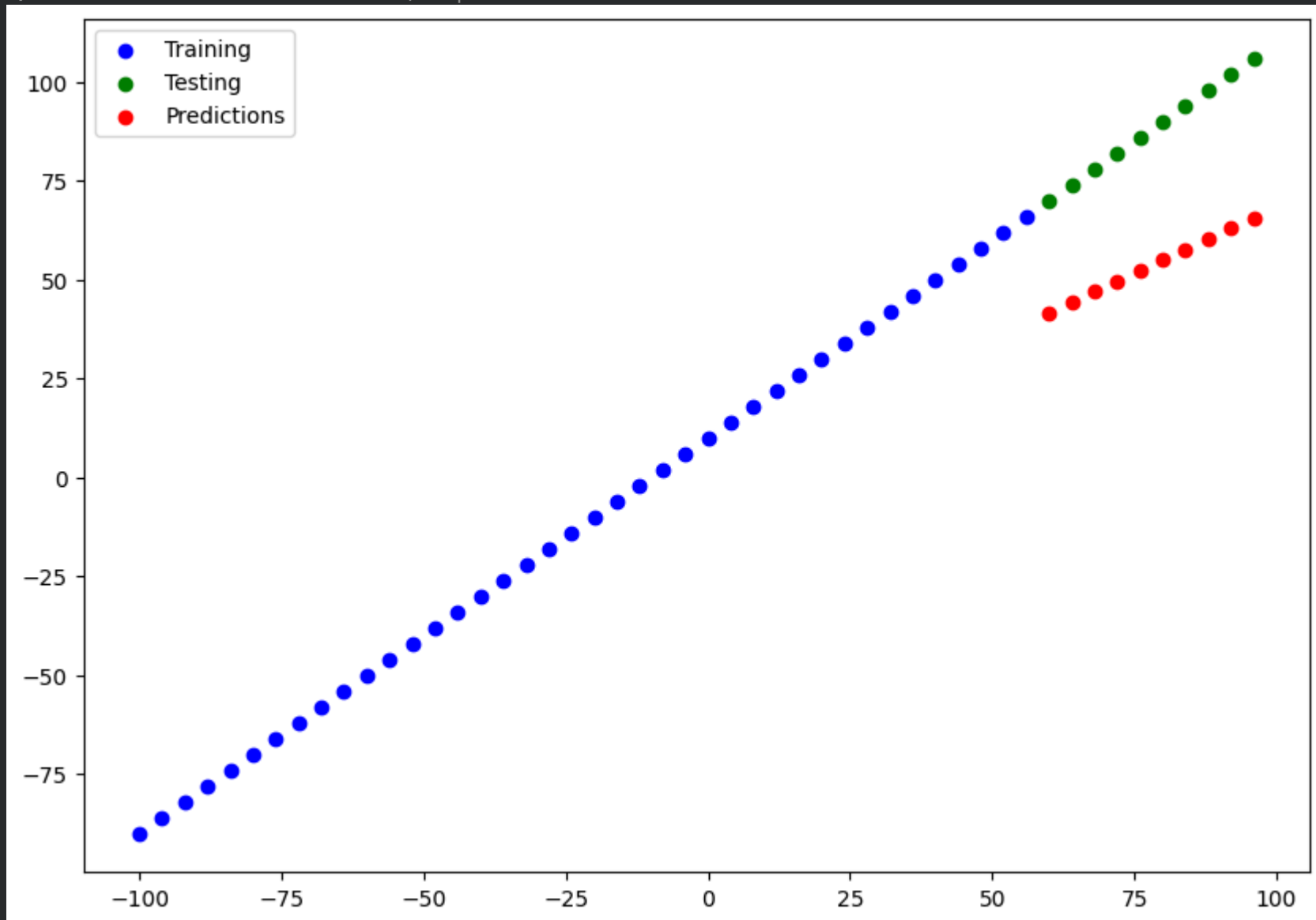
<keras.src.callbacks.history.History at 0x7a170db639e0>
```

```
# visualiza predicciones
y_preds_2 = tf.squeeze(model_2.predict(X_test))
plot_predictions(train_data=X_train,
```



```
train_labels=y_train,
test_data=X_test,
test_labels=y_test,
predictions=y_preds_2)
```

1/1 ————— 0s 114ms/step



.... raro.... hemos añadido una capa y el resultado es mucho peor, ¿quizá entrenando más épocas?

```
# muestra mae y mse
mae_2 = mae(y_test, y_preds_2)
mse_2 = mse(y_test, y_preds_2)
print("METRICAS model_2:\nMAE:",mae_2.numpy(),"\nMSE:",mse_2.numpy())
```

```
METRICAS model_2:
MAE: 34.365856
MSE: 1195.6893
```

Construimos el **modelo 3** (*model_3*) que es igual, pero entrenando más épocas

```
# construye el model_3 (Clon de model_2) y entrénalo 500 épocas
tf.random.set_seed(42)

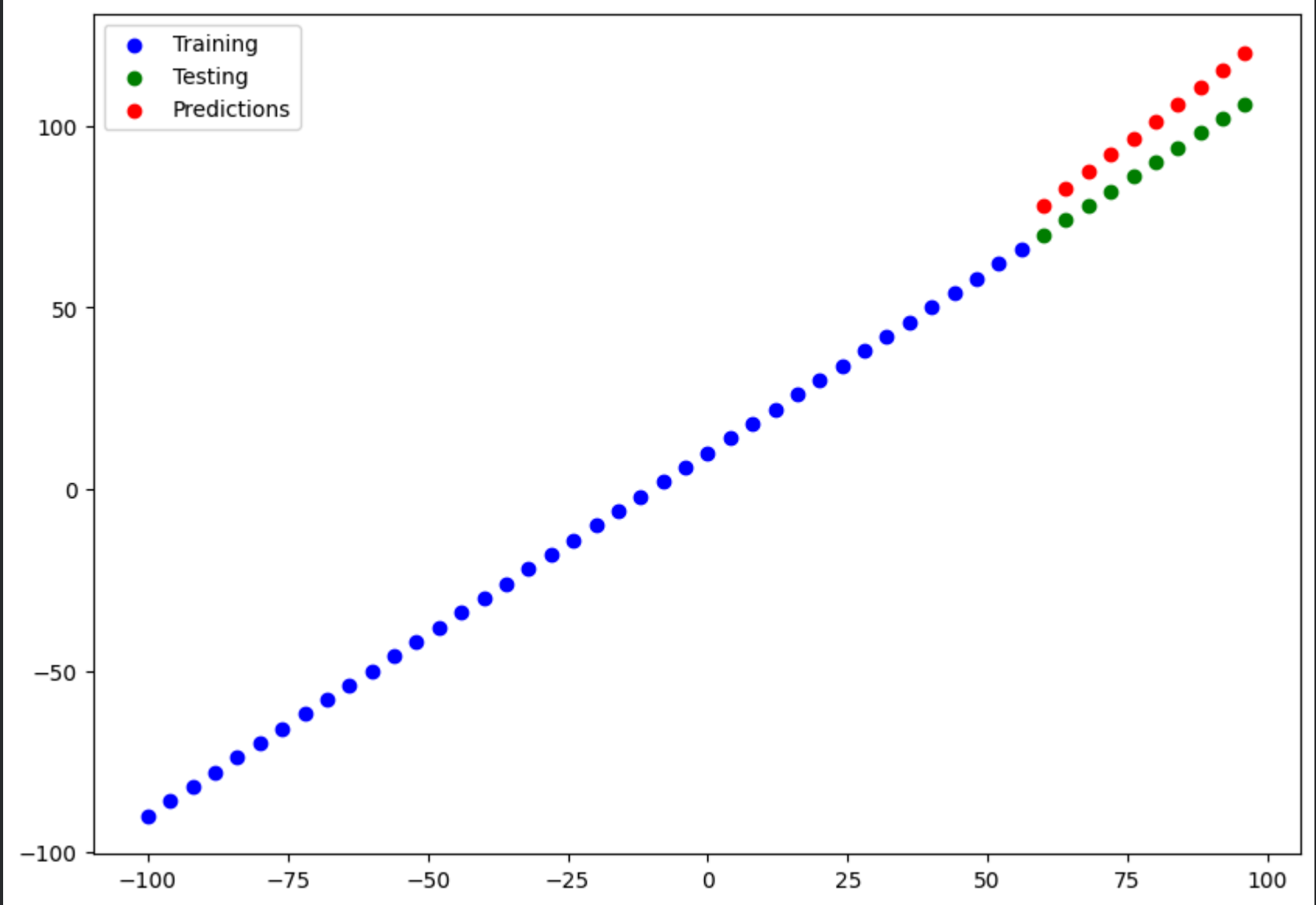
model_3 = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=[1]),
    tf.keras.layers.Dense(1)
])

model_3.compile(loss="mae",
                optimizer="SGD",
                metrics=["mae"])

model_3.fit(X_train, y_train, epochs=500, verbose=0)
```

```
<keras.src.callbacks.history.History at 0x7a170d9313d0>
```

```
# realiza predicciones
y_preds_3 = tf.squeeze(model_3.predict(X_test))
plot_predictions(train_data=X_train,
                 train_labels=y_train,
                 test_data=X_test,
                 test_labels=y_test,
                 predictions=y_preds_3)
```



```
# muestra mae y mse
mae_3 = mae(y_test, y_preds_3)
mse_3 = mse(y_test, y_preds_3)
print(f"model_3 -> MAE: {mae_3.numpy():.2f} | MSE: {mse_3.numpy():.2f}")
```

```
model_3 -> MAE: 11.04 | MSE: 125.97
```

```
# crea un dataframe de pandas que aglutine los resultados de los experimentos
import pandas as pd
```

```
resultados = pd.DataFrame({
    "modelo": ["model_1", "model_2", "model_3"],
    "mae": [mae_1.numpy(), mae_2.numpy(), mae_3.numpy()],
    "mse": [mse_1.numpy(), mse_2.numpy(), mse_3.numpy()]
})
```

```
resultados
```

	modelo	mae	mse
0	model_1	32.024178	1037.475220
1	model_2	34.365856	1195.689331
2	model_3	11.044931	125.971436

Pasos siguientes:

[Generar código con resultados](#)

[New interactive sheet](#)

A partir de nuestros experimentos, parece que `model_3` fue el que mejor rendimiento tuvo.

Y ahora, probablemente estés pensando, "comparar modelos es un rollo..." y definitivamente puede serlo, aquí solo hemos comparado 3 modelos.

Pero esto es parte de lo que implica el modelado en machine learning: probar muchas combinaciones diferentes de modelos y ver cuál funciona mejor.

Cada modelo que construyes es un pequeño experimento.

🔑 Nota: Uno de tus principales objetivos debe ser minimizar el tiempo entre tus experimentos. Cuantos más experimentos realices, más cosas descubrirás que no funcionan y, a su vez, estarás más cerca de descubrir lo que sí funciona. Recuerda el lema del practicante de machine learning: "experimentar, experimentar, experimentar".

Otra cosa que también descubrirás es que lo que pensabas que funcionaría (como entrenar un modelo por más tiempo) puede no siempre funcionar, y a menudo ocurre lo contrario.

✓ Seguimiento de tus experimentos

Un muy buen hábito que puedes adoptar es hacer un seguimiento de tus experimentos de modelado para ver cuáles funcionan mejor que otros.

Hemos hecho una versión simple de esto arriba (guardando los resultados en diferentes variables).

-  **Recurso:** Pero a medida que construyas más modelos, querrás considerar el uso de herramientas como:
- [TensorBoard](#) - un componente de la biblioteca TensorFlow para ayudar a hacer un seguimiento de los experimentos de modelado (veremos esto más adelante).
 - [Weights & Biases](#) - una herramienta para hacer seguimiento de todo tipo de experimentos de machine learning (la buena noticia es que Weights & Biases se integra con TensorBoard).

Optimización automática de hiperparámetros:

- [Optuna](#) - emplea estrategias de búsqueda como la optimización bayesiana para seleccionar automáticamente los hiperparámetros óptimos, lo que mejora el rendimiento de los modelos sin necesidad de probar manualmente múltiples combinaciones.

Guardando un modelo

Una vez que has entrenado un modelo y encuentras uno que funciona a tu gusto, probablemente querrás guardarlo para usarlo en otro lugar (como una aplicación web o dispositivo móvil).

Puedes guardar un modelo de TensorFlow/Keras usando `model.save()`.

Hay dos formas de guardar un modelo en TensorFlow:

1. El formato Keras (nativo) [Keras](#) (por defecto).
2. El formato [HDF5](#).

La principal diferencia entre los dos es que el formato SavedModel es capaz de guardar automáticamente objetos personalizados (como capas especiales) sin modificaciones adicionales al cargar el modelo de nuevo.

¿Cuál deberías usar?

Dependerá de tu situación, pero el formato SavedModel será suficiente la mayoría de las veces.

Ambos métodos utilizan la misma llamada de método.

```
# guarda el modelo 3 en formato keras
model_3.save("modelo_KERAS.keras")

# guarda el modelo 3 en formato h5 (legacy)
model_3.save("modelo_H5.h5")

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is
```

Para cargar un modelo, tan sólo tenemos que usar la función load:

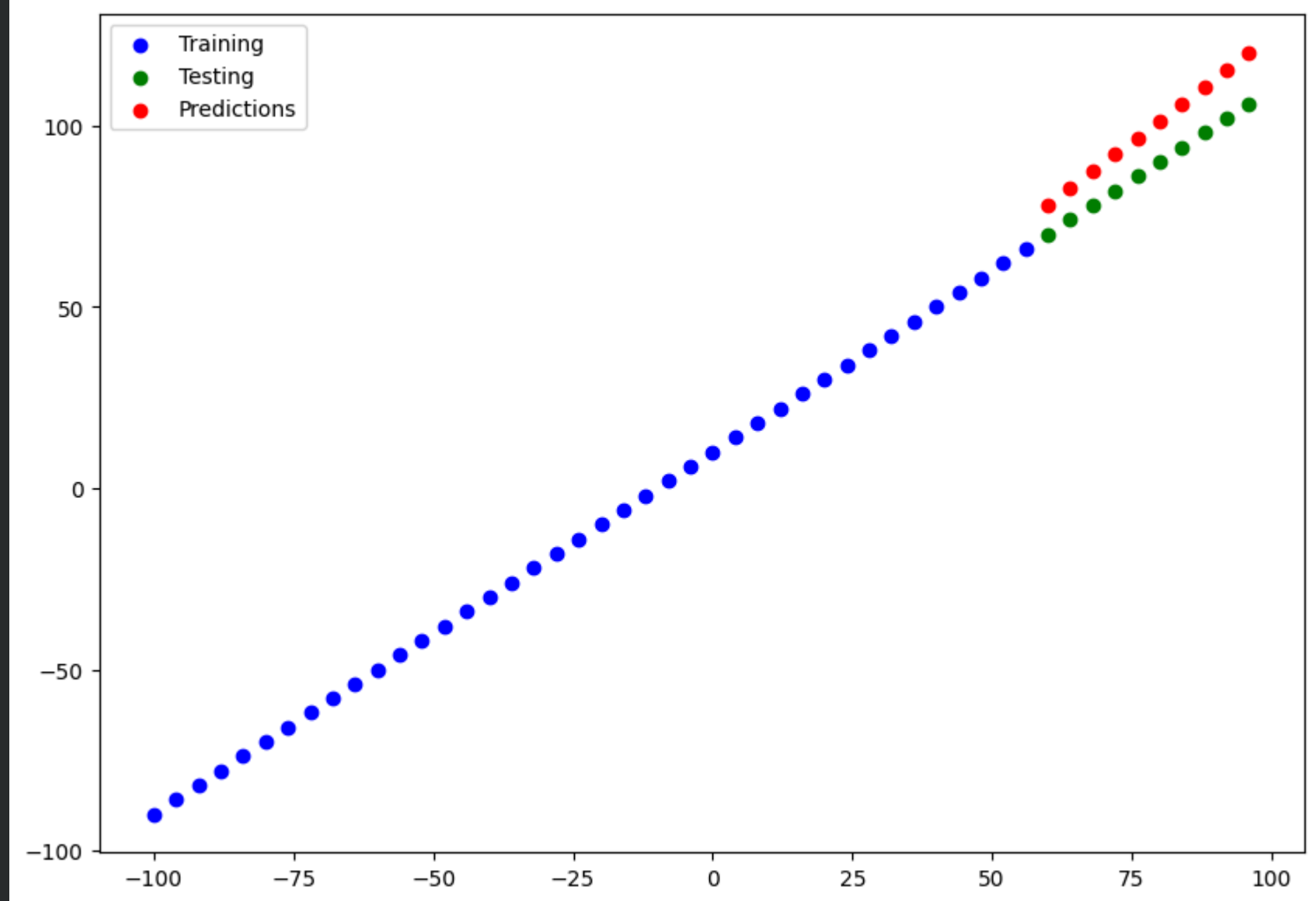
```
# carga el modelo y muestra su summary
modelo_cargado = tf.keras.models.load_model("modelo_KERAS.keras")
modelo_cargado.summary()
```

Model: "sequential_30"

Layer (type)	Output Shape	Param #
dense_38 (Dense)	(None, 1)	2
dense_39 (Dense)	(None, 1)	2

Total params: 6 (28.00 B)
Trainable params: 4 (16.00 B)
Non-trainable params: 0 (0.00 B)
Optimizer params: 2 (12.00 B)

```
# utiliza el modelo cargado para realizar predicciones
y_preds_cargado = tf.squeeze(modelo_cargado.predict(X_test))
plot_predictions(train_data=X_train,
                  train_labels=y_train,
                  test_data=X_test,
                  test_labels=y_test,
                  predictions=y_preds_cargado)
```



Empieza a programar o a [crear código](#) con IA.